

# E-commerce Pipeline Documentation

## Introduction

The E-commerce Pipeline is a fully automated workflow for processing e-commerce transactional data. It validates and transforms the data, calculates key performance indicators (KPIs), and saves the results in DynamoDB. The pipeline is triggered by S3 events, uses ECS Fargate tasks for data validation and transformation, archives data in S3, and sends notifications through SNS. AWS Step Functions orchestrates the entire process, providing reliability, error handling, and monitoring capabilities.

## Goals

- **Validate Data:** Confirm that e-commerce data (products, orders, order items) meets required standards.
- **Transform Data:** Calculate category-level and order-level KPIs from validated data.
- **Store Results:** Save KPIs in DynamoDB for further analysis.
- **Archive Data:** Move processed or invalid data to specific S3 prefixes for storage.
- **Notify Stakeholders:** Send success or failure notifications via SNS.

## Highlights

- **Serverless Design:** Uses AWS Step Functions, ECS Fargate, Lambda, S3, SNS, and DynamoDB for a serverless setup.
- **Retry Mechanism:** Includes retry policies for ECS tasks and Lambda functions to manage temporary failures.
- **Error Management:** Handles errors effectively with fallback paths for invalid data and failure notifications.
- **Monitoring:** Uses CloudWatch to log and monitor activities.

---

## System Architecture

### Architecture Overview

The E-commerce Pipeline consists of several components that work together to process data seamlessly:

#### 1. S3 Bucket (<s3-bucket-name>):

- Holds incoming e-commerce data in the landing-data/ prefix.

- Archives data in archive/processed-data/ for successful runs or archive/invalid-data/ for failed validations.
- Initiates the pipeline through S3 events.

**2. SQS Queue (<sqs-queue-name>):**

- Captures S3 event notifications for new files in landing-data/.
- Ensures events are reliably delivered to the pipeline.

**3. Lambda Function (<s3-event-lambda-name>):**

- Monitors the SQS queue for S3 events.
- Starts the Step Functions state machine.

**4. Step Functions State Machine (EcommercePipeline):**

- Manages the entire workflow.
- Coordinates validation, transformation, data archiving, and notifications.
- Supports retries for temporary failures and error handling for invalid data.

**5. ECS Fargate Tasks:**

- Validation Task (<validation-task-name>): Checks the incoming data using a containerized application.
- Transformation Task (<transformation-task-name>): Processes validated data, calculates KPIs, and stores them in DynamoDB.

**6. DynamoDB Tables:**

- CategoryKPIs: Stores KPIs at the category level.
- OrderKPIs: Stores KPIs at the order level.

**7. Lambda Function (<move-s3-files-lambda-name>):**

- Moves files between S3 prefixes (e.g., from landing-data/ to archive/processed-data/ or archive/invalid-data/).

**8. SNS Topic (<sns-topic-name>):**

- Sends notifications about pipeline success or failure.
- Delivers messages to subscribed endpoints like email or SMS.

**9. CloudWatch Logs:**

- Records logs for ECS tasks, Lambda functions, and Step Functions for monitoring and debugging.

### Workflow Diagram:

- S3 (landing-data/) → SQS → Lambda (<s3-event-lambda-name>) → Step Functions (EcommercePipeline)
  - Step Functions:
    - Executes RunValidationTask (ECS Fargate).
    - If validation fails → MoveToInvalidData (Lambda) → NotifyInvalidDataMoved (SNS) → Fail.
    - If validation succeeds → RunTransformationTask (ECS Fargate) → MoveToProcessedData (Lambda) → NotifySuccess (SNS) → Success.
  - Data is archived in S3 (archive/processed-data/ or archive/invalid-data/).
  - KPIs are saved in DynamoDB (CategoryKPIs, OrderKPIs).
  - Notifications are sent via SNS.
- 

### Workflow Details

**The pipeline is managed by an AWS Step Functions state machine called EcommercePipeline. The state machine definition is located in the repository at scripts/step\_function.json. Here's how the workflow operates:**

#### 1. Initiation:

- The process begins when files are uploaded to the landing-data/ prefix in the S3 bucket.
- S3 events are sent to an SQS queue, which is monitored by a Lambda function (<s3-event-lambda-name>).
- The Lambda function starts the EcommercePipeline state machine.

#### 2. Validation (RunValidationTask):

- An ECS Fargate task (<validation-task-name>) checks the data for issues like missing columns or incorrect data types.
- If the data passes validation, the workflow moves to RunTransformationTask.
- If validation fails, it proceeds to MoveToInvalidData.

### **3. Move Invalid Data (MoveToInvalidData):**

- A Lambda function (<move-s3-files-lambda-name>) shifts the invalid data from landing-data/ to archive/invalid-data/.
- The outcome is evaluated in CheckMoveToInvalidDataResult.

### **4. Check Move Result (CheckMoveToInvalidDataResult):**

- A Choice state reviews the Lambda response:
  - If the statusCode is 200 (success), it moves to NotifyInvalidDataMoved.
  - If not, it goes to NotifyFailure.

### **5. Notify Invalid Data Moved (NotifyInvalidDataMoved):**

- Sends an SNS notification stating that the pipeline failed during validation and the data was archived in archive/invalid-data/.
- Proceeds to Fail.

### **6. Transformation (RunTransformationTask):**

- An ECS Fargate task (<transformation-task-name>) processes the validated data and computes KPIs.
- Saves the KPIs to DynamoDB (CategoryKPIs and OrderKPIs).
- If successful, moves to MoveToProcessedData.
- If it fails, goes to NotifyFailure.

### **7. Move Processed Data (MoveToProcessedData):**

- A Lambda function (<move-s3-files-lambda-name>) transfers the processed data from landing-data/ to archive/processed-data/.
- If successful, proceeds to NotifySuccess.
- If it fails, moves to NotifyFailure.

### **8. Notify Success (NotifySuccess):**

- Sends an SNS notification confirming that the pipeline completed successfully and KPIs were saved in DynamoDB.
- Moves to Success.

### **9. Notify Failure (NotifyFailure):**

- Sends an SNS notification detailing the failure, including the name of the failed state (e.g., RunValidationTask, RunTransformationTask) and the error message.
- Proceeds to Fail.

## **10. Final States:**

- Success: The pipeline completed successfully.
- Fail: The pipeline failed due to invalid data or an error.

---

## **Setup and Deployment**

**This section provides straightforward steps to set up the E-commerce Pipeline using the AWS Management Console. You'll use the command line to build and push Docker images to Amazon ECR, but all other AWS configurations will be done via the console. These steps assume you have the repository cloned on your local machine.**

### **Requirements**

- AWS Account: Must have permissions to create and manage S3, SQS, Lambda, Step Functions, ECS, ECR, DynamoDB, SNS, and CloudWatch resources.
- AWS CLI: Installed and configured with credentials to push Docker images to ECR.
- Docker: Installed on your local machine for building Docker images.
- Git: To clone the repository.
- Repository: Clone the project repository, which includes:
  - scripts/step\_function.json: Step Functions state machine definition.
  - scripts/lambda\_functions/CheckS3AndTriggerEcommercePipeline.py: Code for the S3 event Lambda function.
  - docker-images/validation/: Code and Dockerfile for the validation task.
  - docker-images/transformation/: Code and Dockerfile for the transformation task.
  - test-data/: Sample data for testing (e.g., products.csv, orders/order1.csv, order\_items/order\_items\_part1.csv).

### **Step 1: Clone the Repository**

1. Use Git to clone the repository to your local machine.
2. Navigate to the project directory.

## **Step 2: Build Docker Images and Push to ECR**

### **2.1 Set Up ECR Repositories**

1. In the AWS Management Console, go to ECR.
2. Click Create repository:
  - Name: validation-task.
  - Create the repository.
3. Repeat to create another repository named transformation-task.
4. Note the repository URIs (e.g., <account-id>.dkr.ecr.<region>.amazonaws.com/validation-task).
5. Authenticate Docker to your ECR registry using the AWS CLI:
  - Run this command in your terminal:
  - `aws ecr get-login-password --region <region> | docker login --username AWS --password-stdin <account-id>.dkr.ecr.<region>.amazonaws.com`
  - Replace <region> with your AWS region (e.g., eu-west-1) and <account-id> with your AWS account ID.

### **2.2 Build and Push the Validation Task Docker Image**

1. In your terminal, navigate to the validation task directory:
2. `cd docker-images/validation`
  - This directory should contain the application code and a Dockerfile.
3. Build the Docker image:
4. `docker build -t validation-task .`
5. Tag the image for ECR:
6. `docker tag validation-task:latest <account-id>.dkr.ecr.<region>.amazonaws.com/validation-task:latest`
7. Push the image to ECR:
8. `docker push <account-id>.dkr.ecr.<region>.amazonaws.com/validation-task:latest`

## 2.3 Build and Push the Transformation Task Docker Image

1. Navigate to the transformation task directory:
2. `cd ../transformation`
  - This directory should contain the application code and a Dockerfile.
3. Build the Docker image:
4. `docker build -t transformation-task .`
5. Tag the image for ECR:
6. `docker tag transformation-task:latest <account-id>.dkr.ecr.<region>.amazonaws.com/transformation-task:latest`
7. Push the image to ECR:
8. `docker push <account-id>.dkr.ecr.<region>.amazonaws.com/transformation-task:latest`

### Step 3: Create IAM Roles

1. Go to IAM > Roles > Create role.
2. Step Functions Execution Role:
  - Choose AWS service > Step Functions as the trusted entity.
  - Attach policies:
    - Permissions for ECS (ecs:RunTask, ecs:StopTask, ecs:DescribeTasks).
    - Permissions for Lambda (lambda:InvokeFunction).
    - Permissions for SNS (sns:Publish).
    - Permissions for CloudWatch Logs (logs:\*)
  - Name the role StepFunctions-EcommercePipeline-Role.
3. ECS Task Execution Role:
  - Choose AWS service > Elastic Container Service > Elastic Container Service Task.
  - Attach policies:
    - Permissions to pull images from ECR (ecr:BatchGetImage, ecr:GetDownloadUrlForLayer).

- Permissions to write logs to CloudWatch (logs:\*)).
    - Permissions to write to DynamoDB (dynamodb:PutItem).
  - Name the role ecsTaskExecutionRole.
4. Lambda Execution Role:
- Choose AWS service > Lambda.
  - Attach policies:
    - Permissions to poll SQS (sqs:ReceiveMessage, sqs:DeleteMessage, sqs:GetQueueAttributes).
    - Permissions to start Step Functions executions (states:StartExecution).
    - Permissions to access S3 (s3:\*)).
    - Permissions to write logs to CloudWatch (logs:\*)).
  - Name the role LambdaExecutionRole.

#### **Step 4: Create S3 Bucket**

1. Go to S3 > Create bucket.
2. Name it <s3-bucket-name> and select your region (e.g., eu-west-1).
3. Create folders:
  - Click Create folder and name it landing-data.
  - Create folders archive/processed-data and archive/invalid-data.

#### **Step 5: Create SQS Queue**

1. Go to SQS > Create queue.
2. Select Standard Queue, name it <sqs-queue-name>, and create it.
3. Go to the S3 bucket > Properties > Event notifications > Create event notification.
  - Name: LandingDataUpload.
  - Event types: Select All object create events.
  - Prefix: landing-data/.
  - Destination: SQS queue, select <sqs-queue-name>.
4. In the SQS queue, go to Access policy and allow S3 to send messages:



- Edit the policy to allow s3.amazonaws.com to perform sqs:SendMessage on this queue, with the condition that the source is your S3 bucket.

## **Step 6: Deploy Lambda Functions**

### **1. S3 Event Lambda (CheckS3AndTriggerEcommercePipeline):**

- Go to Lambda > Create function.
- Name: CheckS3AndTriggerEcommercePipeline.
- Runtime: Python 3.9 (or the version compatible with the script).
- Role: Select LambdaExecutionRole.
- Upload the code:
  - Go to the scripts/lambda\_functions directory on your local machine.
  - Zip the CheckS3AndTriggerEcommercePipeline.py file.
  - In the Lambda console, upload the zip file.
- Add a trigger:
  - Go to Add trigger > Select SQS.
  - Choose <sqs-queue-name>.

### **2. Move S3 Files Lambda (<move-s3-files-lambda-name>):**

- Go to Lambda > Create function.
- Name: <move-s3-files-lambda-name>.
- Runtime: Node.js 18.x.
- Role: Select LambdaExecutionRole.
- Upload the code:
  - Go to the scripts/lambda\_functions directory on your local machine.
  - Zip the contents of the move-s3-files-lambda folder (if available, or adjust based on your repository).
  - Upload the zip file in the Lambda console.

## **Step 7: Create ECS Cluster and Task Definitions**

### **1. Create the ECS Cluster:**

- Go to ECS > Clusters > Create cluster.
- Name: <cluster-name>.
- Infrastructure: Select AWS Fargate (serverless).
- Create the cluster.

## 2. Create Task Definitions:

- Go to ECS > Task definitions > Create new task definition.
- Validation Task:
  - Name: validation-task.
  - Infrastructure: Fargate.
  - CPU: 0.25 vCPU, Memory: 0.5 GB.
  - Task role and execution role: ecsTaskExecutionRole.
  - Add a container:
    - Name: validation-task.
    - Image: <account-id>.dkr.ecr.<region>.amazonaws.com/validation-task:latest.
    - Enable logging: Select CloudWatch Logs, log group /ecs/validation-task.
  - Create the task definition.
- Transformation Task:
  - Repeat the steps above.
  - Name: transformation-task.
  - Image: <account-id>.dkr.ecr.<region>.amazonaws.com/transformation-task:latest.
  - Log group: /ecs/transformation-task.

## 3. Create CloudWatch Log Groups:

- Go to CloudWatch > Log groups > Create log group.
- Create two log groups: /ecs/validation-task and /ecs/transformation-task.

## Step 8: Create DynamoDB Tables

1. Go to DynamoDB > Tables > Create table.
2. CategoryKPIs:
  - Table name: CategoryKPIs.
  - Partition key: Category (String).
  - Use default settings for capacity.
3. OrderKPIs:
  - Table name: OrderKPIs.
  - Partition key: OrderId (String).
  - Use default settings.

### **Step 9: Create SNS Topic**

1. Go to SNS > Topics > Create topic.
2. Name: <sns-topic-name>.
3. Create a subscription:
  - Protocol: Email.
  - Endpoint: Your email address.
  - Confirm the subscription via the email you receive.

### **Step 10: Create Step Functions State Machine**

1. Go to Step Functions > State machines > Create state machine.
2. Name: EcommercePipeline.
3. Use the Code option:
  - Open scripts/step\_function.json from the repository.
  - Replace placeholders with actual values (e.g., ARNs for the cluster, task definitions, Lambda functions, SNS topic, subnets, security groups).
  - Copy and paste the updated JSON into the console.
4. Role: Select StepFunctions-EcommercePipeline-Role.
5. Enable logging:
  - Go to Logging > Enable CloudWatch Logs.
  - Log level: ALL.

- Create a new log group: /aws/states/EcommercePipeline.

## Step 11: Test the Pipeline

### 1. Success Scenario:

- Go to S3 > <s3-bucket-name> > landing-data.
- Upload the test files from the repository (test-data/products.csv, test-data/orders/order1.csv, test-data/order\_items/order\_items\_part1.csv).
- Go to Step Functions > EcommercePipeline and check the latest execution.
- Confirm that data is moved to archive/processed-data/, KPIs are in DynamoDB, and you receive an SNS success notification.

### 2. Failure Scenario:

- Upload invalid data (e.g., modify products.csv to remove required columns).
- Verify that the pipeline fails at validation, data is moved to archive/invalid-data/, and you receive an SNS failure notification.

---

## Monitoring and Debugging

### Monitoring

- **Step Functions Execution History:**

- Check execution details in the Step Functions console to monitor the workflow's progress.
- Look for retries, failures, and state transitions.

- **CloudWatch Logs:**

- ECS task logs: /ecs/validation-task and /ecs/transformation-task.
- Lambda logs: /aws/lambda/CheckS3AndTriggerEcommercePipeline and /aws/lambda/<move-s3-files-lambda-name>.
- Step Functions logs: /aws/states/EcommercePipeline.

- **SNS Notifications:**

- Monitor SNS notifications to track success or failure messages.

### Debugging

- **Validation Issues:**
    - Review the ECS task logs for RunValidationTask to understand why data validation failed.
    - Ensure the data matches the expected format.
  - **Transformation Issues:**
    - Check the ECS task logs for RunTransformationTask to troubleshoot transformation errors.
    - Verify DynamoDB permissions and table structure.
  - **Lambda Problems:**
    - Inspect the Lambda logs for MoveToInvalidData or MoveToProcessedData to identify S3-related issues or throttling.
  - **SNS Notification Problems:**
    - Confirm that the SNS topic has active subscriptions.
    - Ensure the Step Functions role has sns:Publish permissions.
  - **Retry Behavior:**
    - If a task or Lambda fails, review the execution history for retry attempts (up to 3 retries for specific errors).
- 

## Summary

The E-commerce Pipeline offers a reliable, serverless solution for handling e-commerce transactional data, calculating KPIs, and storing them in DynamoDB. Powered by AWS Step Functions, it ensures dependable operation with built-in retry logic, error handling, and monitoring. The setup steps provided make it easy for anyone to deploy the pipeline using the repository, ensuring accessibility for team members.